ЛЕКЦИЯ 4

Основы Python: управляющие конструкции

Информационные технологии и программирование

Условный оператор · Циклы · Вложенные циклы

План лекции

1. Управляющие конструкции в программировании
2. Условный оператор `if`
3. Полная форма: `if-elif-else`
4. Тернарный условный оператор
5. Цикл `while`
6. Цикл `for`
7. Вложенные циклы
8. Операторы `break`, `continue`, `else` в циклах

Три базовые конструкции

В структурном программировании (Бём, Дейкстра, 1966) выделены:

Конструкция	Назначение	Реализация в Python
Следование	Последовательное выполнение	Порядок строк кода
Ветвление	Выбор пути выполнения	`if`, `elif`, `else`
Цикл	Повторение действий	`while`, `for`

> Любая вычислимая функция может быть реализована комбинацией этих трёх конструкций.

Особенности синтаксиса Python

Вложенность блоков определяется отступами, а не фигурными скобками:

```
if condition:
    statement_1     # 4 пробела — блок принадлежит if
    statement_2

# ОШИБКА: неоднородные отступы
if condition:
    statement_1
      statement_2   # IndentationError!
```

- Стандартный отступ: **4 пробела** (PEP 8)
- Двоеточие `:` — начало вложенного блока

Условный оператор `if`

Общая форма

```
if условие:
    блок_операторов
```

Семантика:

1. Вычисляется `условие`
2. Если `True` → выполняется блок
3. Если `False` → блок пропускается

Приведение к `bool`

`False`	`True`
`0`, `0.0`, `""`, `[]`, `{}`, `None`	Любое непустое значение

Пустые коллекции, ноль и None считаются ложными. Это позволяет писать идиоматичный код вроде «if items:» вместо «if len(items) > 0:

Операторы сравнения и логика

```
# Сравнение
==, !=, <, >, <=, >=

# Тождественность (идентичность объектов)
is, is not

# Принадлежность
in, not in

# Логические связки
and, or, not
```

Цепочки сравнений

```
x = 5
if 0 < x < 10:          # эквивалентно: 0 < x and x < 10
    print("x в диапазоне (0, 10)")
```

Ветвление `if-else`

```
if условие:
    блок_А
else:
    блок_Б
```

> Ровно **один** из двух блоков будет выполнен.

Пример

```
number = int(input("Введите целое число: "))

if number % 2 == 0:
    print("Число чётное")
else:
    print("Число нечётное")
```

Множественное ветвление: `if-elif-else`

```
if условие_1:
    блок_1
elif условие_2:
    блок_2
elif условие_3:
    блок_3
else:
    блок_по_умолчанию
```

Алгоритм: проверка условий сверху вниз; выполняется **первый** истинный блок; `else` — если ни одно не истинно.

Пример: `if-elif-else`

```
score = 85

if score >= 90:
    grade = "Отлично"
elif score >= 75:
    grade = "Хорошо"
elif score >= 60:
    grade = "Удовлетворительно"
else:
    grade = "Неудовлетворительно"

print(f"Оценка: {grade}")   # Оценка: Хорошо
```

Конструкция `match-case` (Python 3.10+)

```
command = "quit"

match command:
    case "start":
        print("Запуск процесса")
    case "stop":
        print("Остановка процесса")
    case "quit":
        print("Выход из программы")
    case _:
        print("Неизвестная команда")
```

> Структурное сопоставление с образцом. Альтернатива длинным цепочкам `elif`.

Тернарный условный оператор

Синтаксис

```
значение_если_истина  if  условие  else  значение_если_ложь
```

Пример

```
age = 20
status = "совершеннолетний" if age >= 18 else "несовершеннолетний"
```

Эквивалент через `if-else`

```
if age >= 18:
    status = "совершеннолетний"
else:
    status = "несовершеннолетний"
```

Тернарный оператор: применение

В списковых включениях

```
numbers = [3, -1, 7, -5, 2]
absolute = [x if x >= 0 else -x for x in numbers]
# [3, 1, 7, 5, 2]
```

Ограничения

- ❌ В Python нет формы `условие ? a : b`
- ❌ Не рекомендуется вкладывать тернарные операторы:

```
# Плохо читается:
sign = "positive" if x > 0 else ("negative" if x < 0 else "zero")
```

> Для трёх и более случаев используйте `if-elif-else`.

Цикл `while`

Общая форма

```
while условие:
    тело_цикла
```

Семантика:

1. Вычисляется условие
2. `True` → выполняется тело → возврат к п. 1
3. `False` → цикл завершается

Пример

```
i = 1
while i <= 5:
    print(i)
    i += 1    # без этой строки — бесконечный цикл!
```

Цикл `while`: примеры

Сумма цифр числа

```
n = int(input("Введите число: "))
digit_sum = 0

while n > 0:
    digit_sum += n % 10   # последняя цифра
    n //= 10              # отбрасываем последнюю цифру

print("Сумма цифр:", digit_sum)
```

Намеренный бесконечный цикл

```
while True:
    cmd = input("Команда (exit для выхода): ")
    if cmd == "exit":
        break
    print(f"Выполняю: {cmd}")
```

Цикл `for`

Итерационная модель

```
for переменная in итерируемый_объект:
    тело_цикла
```

> В отличие от C/Java, `for` в Python — это **обход последовательности**, а не цикл со счётчиком.

Примеры

```
for fruit in ["яблоко", "банан", "вишня"]:
    print(fruit)

for char in "Python":
    print(char, end=" ")    # P y t h o n
```

Функция `range()`

```
range(stop)                # 0, 1, 2, ..., stop-1
range(start, stop)         # start, start+1, ..., stop-1
range(start, stop, step)   # start, start+step, ..., < stop
```

```
for i in range(1, 6):
    print(i, end=" ")      # 1 2 3 4 5

for i in range(0, 10, 2):
    print(i, end=" ")      # 0 2 4 6 8

for i in range(10, 0, -1):
    print(i, end=" ")      # 10 9 8 7 6 5 4 3 2 1
```

> ⚠️ `range()` — **ленивый** объект: элементы генерируются по запросу, список в памяти не создаётся.

`enumerate()` и `zip()`

`enumerate()` — индекс + значение

```
languages = ["Python", "C", "Java"]
for i, lang in enumerate(languages, start=1):
    print(f"{i}. {lang}")
# 1. Python
# 2. C
# 3. Java
```

`zip()` — параллельная итерация

```
names  = ["Алиса", "Боб", "Карл"]
scores = [95, 82, 78]
for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

`else` после циклов

Уникальная особенность Python:

```
n = 7
i = 2
while i * i <= n:
    if n % i == 0:
        print(f"{n} делится на {i}")
        break
    i += 1
else:
    print(f"{n} — простое число")
```

Завершение цикла	`else` выполняется?
Условие стало `False` / итерации исчерпаны	✅ Да
Цикл прерван через `break`	❌ Нет

Вложенные циклы

Определение

Цикл, в теле которого содержится другой цикл.

```
for i in range(n):          # внешний цикл
    for j in range(m):      # внутренний цикл
        ...                 # выполняется n × m раз
```

Пример: прямоугольный треугольник

```
for i in range(1, 6):
    for j in range(i):
        print("*", end="")
    print()
```

```
*
**
***
****
*****
```

Вложенные циклы: таблица умножения

```
for i in range(1, 10):
    for j in range(1, 10):
        print(f"{i * j:4d}", end="")
    print()
```

Результат: таблица умножения 9×9.

Пример: поиск пары с заданной суммой

```
numbers = [2, 7, 11, 15, 3, 6]
target = 9

for i in range(len(numbers)):
    for j in range(i + 1, len(numbers)):
        if numbers[i] + numbers[j] == target:
            print(f"Найдено: {numbers[i]} + {numbers[j]} = {target}")
```

Вычислительная сложность

Структура	Сложность
Один цикл по `n` элементов	$O(n)$
Два вложенных цикла по `n`	$O(n^2)$
Три вложенных цикла по `n`	$O(n^3)$
Вложенный `range(i, n)`	$O(n^2/2) \approx O(n^2)$

> ⚠️ При **n > 10 000** двойное вложение может стать критическим по времени.

Примеры

- `n = 100` → 10 000 итераций ✓
- `n = 10 000` → 100 000 000 итераций ⚠️
- `n = 100 000` → 10 000 000 000 итераций ❌

Важно понимать, что вложенные циклы дают квадратичную или кубическую сложность. Для малых данных это незаметно, но при росте объёма время выполнения растёт квадратично. На практике для больших данных ищут алгоритмы с линейной или логарифмической сложностью, используя словари, множества, сортировку.

Операторы `break` и `continue`

`break` — выход из цикла

```
for i in range(100):
    if i == 10:
        break
    print(i, end=" ")    # 0 1 2 3 4 5 6 7 8 9
```

`continue` — пропуск итерации

```
for i in range(10):
    if i % 2 == 0:
        continue
    print(i, end=" ")    # 1 3 5 7 9
```

Операторы `break` и `continue`

Оператор	Действие
`break`	Немедленный выход из цикла
`continue`	Переход к следующей итерации
`pass`	Заглушка (ничего не делает)

break полностью прерывает цикл, управление передаётся за его пределы. **continue** прерывает только текущую итерацию — оставшийся код в теле цикла пропускается, и начинается следующая итерация. Операторы действуют только на тот цикл, в котором непосредственно находятся.

Выход из вложенных циклов

`break` прерывает **только ближайший** цикл.

Способ 1: флаговая переменная

```
found = False
for i in range(n):
    for j in range(m):
        if matrix[i][j] == target:
            found = True
            break       # выход из внутреннего
    if found:
        break           # выход из внешнего
```

Способ 2: функция с `return`

```
def find_target(matrix, target):
    for i in range(len(matrix)):
        for j in range(len(matrix[i])):
            if matrix[i][j] == target:
                return (i, j)
    return None
```

Сводная таблица конструкций

Конструкция	Назначение	Пример
`if`	Условное выполнение	`if x > 0: ...`
`if-else`	Выбор из двух ветвей	`if x > 0: ... else: ...`
`if-elif-else`	Множественный выбор	`if ... elif ... else ...`
Тернарный	Краткий выбор значения	`a if cond else b`
`while`	Цикл с условием	`while x > 0: ...`
`for`	Обход последовательности	`for x in items: ...`
`break`	Выход из цикла	внутри `while` / `for`
`continue`	Следующая итерация	внутри `while` / `for`

Разбор примеров

Что выведет программа?

Пример 1. Сравнение строк

Что выведет программа?

```
a = "яблоко"
b = "арбуз"

if a > b:
    print("Первое больше")
else:
    print("Второе больше")
```

Пример 1. Ответ

```
a = "яблоко"
b = "арбуз"

if a > b:
    print("Первое больше")   # ← выполнится
else:
    print("Второе больше")
```

Вывод: `Первое больше`

Пояснение: Строки сравниваются **посимвольно** по кодам символов (таблица Unicode). Код буквы `я` (1103) больше кода буквы `а` (1072), поэтому `"яблоко" > "арбуз"` → `True`. Сравнение происходит по первому же несовпадающему символу.

Пример 2. Тернарный оператор

Что выведет программа?

```
x = 15
result = "чётное" if x % 2 == 0 else "нечётное" if x % 2 != 0 else "ноль"
print(result)
```

Пример 2. Ответ

```
x = 15
result = "чётное" if x % 2 == 0 else "нечётное" if x % 2 != 0 else "ноль"
#          ↑ не выполнится         ↑ выполнится
print(result)
```

Вывод: `нечётное`

Пояснение: Выражение читается как цепочка:

1. `x % 2 == 0` → `15 % 2 == 0` → `False` → переходим в `else`
2. `x % 2 != 0` → `15 % 2 != 0` → `True` → результат `"нечётное"`

Третья ветка (`"ноль"`) никогда не будет достигнута для ненулевых чисел.

Пример 3. Цикл `while`

Что выведет программа?

```
n = 12345
s = 0

while n > 0:
    s += n % 10
    n //= 10

print(s)
```

Пример 3. Ответ

Вывод: `15`

Пояснение: Программа вычисляет **сумму цифр** числа 12345.

Итерация	`n`	`n % 10`	`s`
1	12345	5	5
2	1234	4	9
3	123	3	12
4	12	2	14
5	1	1	**15**

После `n //= 10` на последней итерации `n = 0`, цикл завершается.

Пример 4. Цикл `for` и `range`

Что выведет программа?

```
for i in range(3):
    print(i, end=" ")

print()

for i in range(5, 0, -2):
    print(i, end=" ")
```

Пример 4. Ответ

Вывод:

```
0 1 2
5 3 1
```

Пояснение:

- `range(3)` → `0, 1, 2` (правая граница **не включается**)
- `range(5, 0, -2)` → старт `5`, шаг `-2`, остановка перед `0`:
  - `5` → `5 - 2 = 3` → `3 - 2 = 1` → `1 - 2 = -1 < 0` → **стоп**

Правая граница в `range` всегда **исключается**, независимо от направления шага.

Пример 5. Вложенные циклы и `break`

Что выведет программа?

```
for i in range(3):
    for j in range(3):
        if i == j:
            break
        print(f"({i},{j})", end=" ")
    print()
```

Пример 5. Ответ

Вывод:

```
(1,0)
(2,0) (2,1)
```

Пояснение по итерациям:

`i`	`j`	`i == j`?	Действие
0	0	✅ `break`	Внутренний цикл прерван сразу, вывод пуст
1	0	❌	Печать `(1,0)`
1	1	✅ `break`	Прерывание

`break` выходит **только из внутреннего** цикла. Внешний продолжает работать.

Пример 6. `else` после цикла

Что выведет программа?

```
for n in range(2, 8):
    for x in range(2, n):
        if n % x == 0:
            print(f"{n} = {x} * {n // x}")
            break
    else:
        print(f"{n} — простое число")
```

Пример 6. Ответ

Вывод:

```
2 — простое число
3 — простое число
4 = 2 * 2
5 — простое число
6 = 2 * 3
7 — простое число
```

Пояснение:

- Если внутренний цикл **нашёл делитель** → `break` → блок `else` **пропускается**
- Если внутренний цикл **завершился без** `break` (делителей нет) → выполняется `else`

Для `n=4`: `4 % 2 == 0` → печать разложения → `break` → `else` пропущен.
Для `n=5`: диапазон `range(2, 5)` → ни одно `x` не делит 5 → `else` выполнен.

Пример 7. `continue` в цикле

Что выведет программа?

```
for i in range(10):
    if i % 3 == 0:
        continue
    print(i, end=" ")
```

Пример 7. Ответ

Вывод:

```
1 2 4 5 7 8
```

Пояснение:

- `continue` пропускает **текущую итерацию**, но цикл продолжается
- Пропущены числа, кратные 3: `0, 3, 6, 9`
- Остальные числа печатаются

`i`	`i % 3 == 0`?	Действие
0	✅	`continue` — пропуск
1	❌	печать `1`

Функции в Python

Объявление функции

```
def имя_функции(параметры):
    """Строка документации (docstring)"""
    тело_функции
    return результат

### Пример

def greet(name):
    """Приветствует пользователя по имени."""
    return f"Привет, {name}!"

message = greet("Алиса")
print(message)   # Привет, Алиса!
```

Параметры и аргументы

Позиционные и именованные аргументы

```
def power(base, exponent):
    return base ** exponent

power(2, 3)            # позиционные: base=2, exponent=3
power(exponent=3, base=2)  # именованные: порядок не важен
```

Значения по умолчанию

```
def greet(name, greeting="Привет"):
    return f"{greeting}, {name}!"

greet("Боб")                  # Привет, Боб!
greet("Боб", "Здравствуйте") # Здравствуйте, Боб!
```

> ⚠️ Значения по умолчанию вычисляются **один раз** при определении функции.

Произвольное число аргументов

`*args` — позиционные аргументы

```python
def sum_all(*args):
    total = 0
    for x in args:
        total += x
    return total

sum_all(1, 2, 3)        # 6
sum_all(1, 2, 3, 4, 5) # 15
```

`**kwargs` — именованные аргументы

```python
def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Алиса", age=25, city="Москва")
```

Возврат значений: `return`

```
def divide(a, b):
    if b == 0:
        return None        # ранний выход
    return a / b

result = divide(10, 3)
print(result)   # 3.3333...
```

Несколько возвращаемых значений

```
def min_max(numbers):
    return min(numbers), max(numbers)

lo, hi = min_max([4, 2, 8, 1])
print(lo, hi)   # 1 8
```

> Если `return` отсутствует или вызван без значения, функция возвращает `None`.

Области видимости: правило LEGB

Python ищет имя переменной по уровням:

Уровень	Название	Пример
L	Local	внутри функции
E	Enclosing	во вложенных функциях
G	Global	в модуле
B	Built-in	встроенные (`len` , `print`)

Пример

```
x = 10          # Global

def outer():
    x = 20      # Enclosing
    def inner():
        x = 30  # Local
        print(x)  # 30
    inner()

outer()
```

Ключевые слова `global` и `nonlocal`

`global` — доступ к глобальной переменной

```
counter = 0

def increment():
    global counter
    counter += 1

increment()
print(counter)   # 1
```

`nonlocal` — доступ к переменной объемлющей функции

```
def outer():
    count = 0
    def inner():
        nonlocal count
        count += 1
        return count
    return inner

inc = outer()
print(inc())  # 1
print(inc())  # 2
```

Лямбда-функции

Синтаксис

```
lambda параметры: выражение
```

Примеры

```
square = lambda x: x ** 2
print(square(5))   # 25

add = lambda a, b: a + b
print(add(3, 4))   # 7
```

Использование как аргумент

```
numbers = [3, 1, 4, 1, 5, 9]
sorted_numbers = sorted(numbers, key=lambda x: -x)
print(sorted_numbers)   # [9, 5, 4, 3, 1, 1]
```

Функции высшего порядка

`map()` — применение функции к каждому элементу

```
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x ** 2, numbers))
print(squares)   # [1, 4, 9, 16, 25]
```

`filter()` — фильтрация элементов

```
numbers = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)   # [2, 4, 6]
```

Списковые включения — идиоматичная альтернатива

```
squares = [x ** 2 for x in numbers]
evens = [x for x in numbers if x % 2 == 0]
```

Декораторы (обзор)

Декоратор — функция, изменяющая другую функцию.

```
def timer(func):
    import time
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__}: {time.time() - start:.4f}c")
        return result
    return wrapper

@timer
def slow_function():
    import time
    time.sleep(1)

slow_function()   # slow_function: 1.0012c
```

Документирование функций

Docstring и аннотации типов

```
def rectangle_area(width: float, height: float) -> float:
    """Вычисляет площадь прямоугольника.

    Аргументы:
        width: ширина прямоугольника
        height: высота прямоугольника

    Возвращает:
        Площадь прямоугольника
    """
    return width * height

help(rectangle_area)   # вывод документации
```

Сводная таблица: функции

Конструкция	Назначение	Пример
`def`	Объявление функции	`def f(x): ...`
`return`	Возврат значения	`return x * 2`
`*args`	Произвольные позиционные	`def f(*args):`
`**kwargs`	Произвольные именованные	`def f(**kwargs):`
`lambda`	Анонимная функция	`lambda x: x + 1`
`global`	Доступ к глобальной переменной	`global counter`
`nonlocal`	Доступ к переменной объемлющей	`nonlocal count`
`@decorator`	Обёртка над функцией	`@timer`

Пример 1. Область видимости

Что выведет программа?

```
x = 10

def foo():
    x = 20
    print(x)

foo()
print(x)
```

Пример 1. Ответ

Вывод:

```
20
10
```

Пояснение:

- Внутри `foo()` оператор `x = 20` создаёт **новую локальную** переменную `x`
- Глобальная переменная `x = 10` **не изменяется**
- После выхода из функции локальная переменная уничтожается

Чтобы изменить глобальную переменную, нужно использовать `global`:

```
def foo():
    global x
    x = 20
```